

FLOPS (floating point operations per second) the system can execute. For storage systems, the typical performance measures used are:

1. Throughput,
2. IOPS (number of Input/Output Operations Per Second), and
3. Average latency of an operation.

The most common measure of a storage system—be it a simple SATA drive connected directly to your PC, or a complex storage array controller connected through SCSI—is how much maximum throughput it can deliver. This is measured as data transfer per second (megabytes per second or MBps). IOPS is the number of IO operations that can happen per second on the storage device, and is defined as the total number of read/write operations that can be performed in one second on the storage device in question. An IO operation request will take a certain time to complete. This can be specified in terms of average latency of an IO operation. Each IO operation is an IO request for a certain size of IO. In simple terms, an IO request can be a read operation of X bytes or a write operation of Y bytes. Hence, each IO request is characterised by its type and size. Now, we can approximate the throughput/data transferred into and out of storage systems as:

$\text{Throughput} = \text{IOPS} * \text{average latency of IO operation} * \text{average request size}$

The IO request latency is limited by a number of factors, including the physical factors associated with the storage system. Let us consider a trivial example where the storage system is the traditional hard disk. Here the IO latency is limited by the rotational delay of the disk, the seek time of the disk and data transfer latency. The rotational delay is governed by the disk's rotational speed, which is expressed in Revolutions Per Minute (RPM) and indicates the amount of time it takes to get the right sector under the disk head. The seek time is governed by the time it takes for the disk head to position itself on the right track on the disk. Once the disk head is in the right position, it can start reading/writing the data. The delay incurred in reading/writing the data is the data transfer latency. All these three components contribute to the IO request latency.

Depending on the purpose of the storage system, any or all of these performance measures would be significant. For instance, if you are using the storage system for backup/archival purposes, you may be more concerned with overall IO throughput rather than the individual IO request latency. On the other hand, if the purpose is to serve data to a latency-sensitive/client-facing application such as a database, IO request latency would become a significant measure of performance for

the storage system under consideration. Hence, different performance measures are applicable for different workloads being deployed on the storage system.

Coming back to comparing scale-out storage and scale-up storage, each has its own performance benefits. Scale-up storage can offer good IO request latencies, while scale-out can offer good IO throughputs. Since scale-out storage is a distributed system, it can suffer from greater communication costs incurred between the nodes. For instance, consider a distributed file system supported on scale-out storage. Both the data and the meta-data associated with each file can be distributed over different nodes on the storage system. For instance, consider a simple example where you are trying to access a file `'/dir1/file1'`. It is possible that `dir1` is located on `node1` and `file1` is located on `node2`. Hence, the file access operation spans multiple nodes in the scale-out storage system, which leads to greater communication costs and a longer request latency.

Here is a question for our readers to think about. Given that the file access operation can span multiple nodes as in the example mentioned earlier, let us assume that we are trying to delete the file located at `'/dir1/file1'` in this example. This operation spans two nodes. How can we ensure that the operation is atomic, even if one of the nodes fails while the operation is in progress?

My 'must-read book' for this month

This month's must-read book suggestion comes from one of our readers, Ravi Krishnan. He recommends the book 'Unix Filesystems: Evolution, Design, and Implementation' by Steve Pate. This book discusses how various file systems have evolved over time, and gives detailed information on file systems internals. Thanks Ravi, for your suggestion.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group, [Computer Science Interview Training India](http://www.linkedin.com/groups?gid=2339182) at <http://www.linkedin.com/groups?gid=2339182>